

Programação Orientada a Objetos



Aula 8 - Classes Úteis
Prof. Guilherme Guerino



Classes Wrappers

- Classes Wrappers permitem que um tipo primitivo seja manipulado como se fosse um objeto.
- Para cada tipo primitivo, Java fornece uma classe invólucro/embrulho (wrapper) correspondente.

Grupo	Tipo Primitivo	Classe Wrapper
Lógico	boolean	Boolean
Caractere	char	Character
Inteiros	byte	Byte
	short	Short
	int	Integer
Ponto Flutuante	long	Long
	float	Float
	double	Double



Classes Wrappers

- “Embrulhando” um primitivo em um wrapper:
 - `int x = 569;`
 - `Integer iWrap = new Integer(x);`
- “Desembrulhando” um primitivo:
 - `int y = iWrap.intValue();`
 - Cada wrapper possui um `xyzValue()`, sendo xyz o tipo primitivo desejado.



Classes Wrappers

- Então quer dizer que eu sempre tenho que fazer todo esse esforço para trabalhar com primitivos na forma de objetos?
 - Sim, antes da versão 5.0 você fazia todo esse trabalho!
- A partir da versão 5.0, surgiu o conceito de autoboxing:
 - Faz a conversão de primitivos para wrappers automaticamente!



Classes Wrappers

```
public static void main(String[] args) {  
    ArrayList<Integer> numeros = new ArrayList<Integer>();  
    numeros.add(5);  
    numeros.add(20);  
  
    int numero = numeros.get(0);  
    System.out.println(numero);  
  
    Integer valor;  
    valor = 10;  
    System.out.println(valor);  
}
```



Classes Wrappers

- Cada classe wrapper também fornece um método estático para a conversão de Strings para o tipo primitivo correspondente.

Origem	Destino	Método
String	int	Integer.parseInt(String s)
String	float	Float.parseFloat(String s)
String	byte	Byte.parseByte(String s)
String	short	Short.parseShort(String s)
String	long	Long.parseLong(String s)
String	double	Double.parseDouble(String s)



Classe Math

- A classe Math nos proporciona uma série de operações e constantes matemáticas que são facilmente acessadas estaticamente, ou seja, não precisamos instanciar um objeto para podermos usar seus métodos.



Classe Math - Métodos

- Dentro desta classe encontramos vários métodos e categorizamos os mais utilizados da seguinte forma:
 - Máximo e Mínimo;
 - Potências e Raízes;
 - Arredondamentos e Valores Absolutos.



Classe Math - Métodos

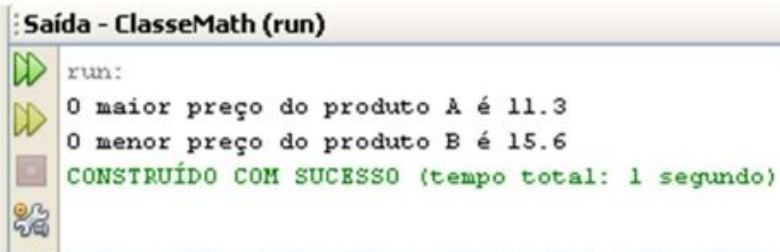
- A classe Math de Java possui dois métodos que retornam o maior e o menor valor de seus dois argumentos.
 - Esses métodos são `max()` e `min()`.



Classe Math - Métodos

```
float precoProdutoA1 = 11.2f;  
float precoProdutoA2 = 11.3f;  
float precoProdutoB1 = 19.7f;  
float precoProdutoB2 = 15.6f;  
System.out.println("O maior preço do produto A é " +  
    Math.max(precoProdutoA1, precoProdutoA2));  
System.out.println("O menor preço do produto B é " +  
    Math.min(precoProdutoB1, precoProdutoB2));
```

► Saída



```
 Saída - ClasseMath (run)  
run:  
0 maior preço do produto A é 11.3  
0 menor preço do produto B é 15.6  
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```



Classe Math - Métodos

- Podemos fazer cálculos de potência e raízes com facilidade usando os métodos disponíveis em Math:
 - `pow (base, expoente)` - calcula a potência da base elevada ao expoente;
 - `sqrt (número)` - calcula a raiz quadrada de um número;
 - `cbrt (número)` - calcula a raiz cúbica de um número;
 - `exp (expoente)` - calcula o valor da constante de Euler (E) elevado ao expoente.



Classe Math - Métodos

```
System.out.println("1MB tem " + Math.pow(2, 10) + " KB");  
System.out.println("A raiz quadrada de 121 é " +  
    Math.sqrt(121));  
System.out.println("A raiz cúbica de 1331 também é " +  
    Math.cbrt(1331));  
System.out.println("O valor da constante de euler elevado a 2 é " +  
    Math.exp(2));
```

Saída

Saída - ClasseMath (run)

```
1MB tem 1024.0 KB  
A raiz quadrada de 121 é 11.0  
A raiz cúbica de 1331 também é 11.0  
O valor da constante de euler elevado a 2 é 7.38905609893065  
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```



Classe Math - Métodos

- Existem algumas formas de arredondar um número fracionado (float e double) transformando-o em um número inteiro e também como obter o valor absoluto de qualquer número:
 - abs (número) - retorna o valor absoluto do mesmo tipo do parâmetro. Exemplo: retorna int positivo, float positivo, etc;
 - ceil (decimal) - este método retorna o valor decimal mais alto do parâmetro. Exemplo: 2.1 será 3;
 - floor (decimal) - este método retorna o valor decimal mais baixo do parâmetro. Exemplo: 2.9 será 2;
 - rint (decimal) - retorna um valor double mais próximo do valor do parâmetro. Exemplo: 2.4 será 2 e 2.9 será 3;
 - round (decimal) - retorna o arredondamento aritmético do número decimal passado como parâmetro. Exemplo: 2.4 será 2 e 2.6 será 3.



Classe Math - Métodos

```
float nr1 = -5.4f;  
float nr2 = -5.5f;  
float nr3 = -5.6f;  
System.out.println("Absoluto: " + Math.abs(nr1) + " / " +  
    Math.abs(nr2) + " / " + Math.abs(nr3));  
System.out.println("Inteiro mais alto: " + Math.ceil(nr1) + " / " +  
    Math.ceil(nr2) + " / " + Math.ceil(nr3));  
System.out.println("Inteiro mais baixo: " + Math.floor(nr1) + " / " +  
    Math.floor(nr2) + " / " + Math.floor(nr3));  
System.out.println("Double mais próximo: " + Math rint(nr1) + " / " +  
    Math rint(nr2) + " / " + Math rint(nr3));  
System.out.println("Arredondamento: " + Math.round(nr1) + " / " +  
    Math.round(nr2) + " / " + Math.round(nr3));
```

Saída

Saída - ClasseMath (run)

```
Inteiro mais alto:-5.0 / -5.0 / -5.0  
Inteiro mais baixo: -6.0 / -6.0 / -6.0  
Double mais próximo: -5.0 / -6.0 / -6.0  
Arredondamento: -5 / -5 / -6  
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```



Classe String

- Uma String em Java não é um tipo primitivo e sim um objeto de uma classe com algumas características especiais.
- Valores literais entre aspas duplas são transformados pelo compilador Java em objetos da classe String.



Classe String - Concatenar

- Concatenação nada mais é do que juntar Strings numa só.
- Isto pode ser feito de duas formas:
 - Usando o método concat() da classe String ou;
 - Usando o sinal de adição (+) como operador de concatenação.



Classe String - Concatenar

```
package classestring;

public class ClasseString {

    public static void main(String[] args) {

        String s1;
        String s2;

        // Métodos para concatenação
        s1 = "Marcos ";
        s2 = "Domingues";
        System.out.println(s1 + s2);
        System.out.println(s1.concat(s2));
    }
}
```

Saída:

↳ Saída - ClasseString (run)



run:



Marcos Domingues

Marcos Domingues



Classe String - Verificar tamanho

- A classe String possui um método que retorna o tamanho da String.
- Esse método é denominado `length()`.



Classe String - Verificar tamanho

```
//Método para verificar o tamanho das Strings  
System.out.println("Tamanho da String 1: " + s1.length());  
System.out.println("Tamanho da String 2: " + s2.length());
```

Saída:

Saída - ClasseString (run)

```
run:  
Tamanho da String 1: 7  
Tamanho da String 2: 9  
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```



Classe String - Pegar caractere

- Podemos também obter um caractere que se encontra em alguma posição dentro da String.
- Para isso, usamos o método `charAt(posição)`.
 - Obs.: Inicia em 0, o primeiro caractere.



Classe String - Pegar caractere

```
//Método que busca qual caractere está na posição indicada  
System.out.println("Caractere na posição 3 da String "  
+ s1 + ": " + s1.charAt(3));  
System.out.println("Caractere na posição 5 da String "  
+ s1 + ": " + s1.charAt(5));
```

Saída:

: Saída - ClasseString (run)



run:



Caractere na posição 3 da String Marcos : c

Caractere na posição 5 da String Marcos : s



CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)





Classe String - Verificar conteúdo

- Um método muito útil para verificar o conteúdo de uma String é o `contains()`.
- O método `contains()` retorna verdadeiro (`true`) se houver a sequência de caracteres especificada no parâmetro.



Classe String - Verificar conteúdo

```
//Método que verifica se a String contém uma sequência de caracteres  
s1 = "Marcos Domingues";  
System.out.println("Contém a string Marcos em s1? "  
+ s1.contains("Marcos"));  
System.out.println("Contém a string Domingo em s1? "  
+ s1.contains("Domingo"));
```

Saída:

```
: Saída - ClasseString (run)  
run:  
Contém a string Marcos em s1? true  
Contém a string Domingo em s1? false  
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```



Classe String - Altera valor para String

- O método `valueOf()` converte um valor para String.

```
int x = 200;  
s1 = String.valueOf(x);  
System.out.println("s1 recebe um valor numérico: " + s1);
```

Saída:

```
 Saída - ClasseString (run)  
run:  
s1 recebe um valor numérico: 200  
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```



Classe String - Outros métodos

- `String replace(char old, char new)` – troca todas as ocorrências de um caractere por outro.
- `String substring(int start, int end)` – retorna parte da String incluindo a posição inicial e excluindo a final.
- `String toUpperCase()` e `String toLowerCase()` – retorna a String em caixa alta e caixa baixa.
- `int indexOf(String)` – retorna a posição inicial da String.
- `char[] toCharArray()` – retorna o vetor de char correspondente a String.



Classe StringBuffer

- É uma sequência de caracteres mutável.
- Otimizada para ser alterada, mas não lida.
- StringBuffer é thread safe, isto é, as operações são realizadas de forma sincronizada.
- StringBuffers podem ser criados através de seus construtores.
 - `StringBuffer buffer1 = new StringBuffer();`
 - `StringBuffer buffer2 = new StringBuffer("Texto inicial");`
 - `StringBuffer buffer3 = new StringBuffer(40);`
- Em Java também existe a classe `StringBuilder` que é semelhante a `StringBuffer` mas não é thread safe



Classe StringBuffer

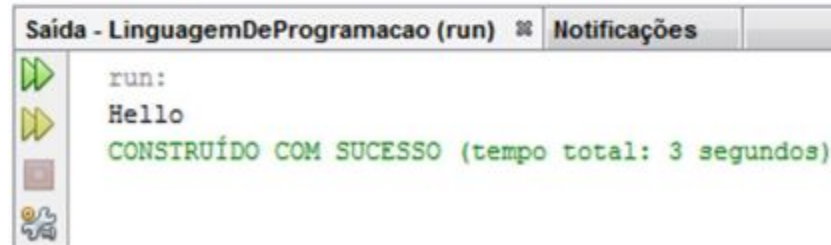
- Métodos de StringBuffer operam sobre o próprio objeto.
 - StringBuffer append(String s) – adiciona texto no final.
 - StringBuffer insert(int posição, String s) – insere texto na posição.
 - void setCharAt(int posição, char c) – substitui o caractere na posição.
 - String toString() – transforma o buffer em String para que possa ser lido.



Classe StringBuffer

```
public static void main(String[] args) {  
    StringBuffer buffer = new StringBuffer("H");  
    buffer.append("e"); buffer.append("l");  
    buffer.append("l"); buffer.append("o");  
    System.out.println(buffer.toString());  
}
```

Saída:





Quando usar String e StringBuffer

- Use String para manipular valores constantes:
 - Textos carregados de fora da aplicação;
 - Valores literais;
 - Textos em geral que não serão modificados intensivamente.
- Use StringBuffer para alterar textos:
 - Acrescentar, concatenar, inserir, etc.
- Prefira usar StringBuffer para construir Strings:
 - Concatenação de strings usando "+" é extremamente cara, pois um novo objeto é criado em cada fase da compilação apenas para ser descartado em seguida;
- Use StringBuffer.append() e, no final, transforme o resultado em String usando o método toString().

Dúvidas?

